

JobPilot: AI-Powered Job Matching Copilot

BAX-423 Final Project Technical Brief

GitHub Repository: <https://github.com/yoyo2809/jobpilot> **Deployment:** <https://jobpilot-evj8gpsfhjxmav2ugmbdb.streamlit.app/>

1. Executive Summary & Product Goal

JobPilot is an adaptive job recommendation engine designed to bridge the semantic gap between a candidate's unstructured resume and complex job descriptions. Unlike traditional keyword-based applicant tracking systems (ATS), JobPilot leverages dense vector embeddings, a multi-stage ranking pipeline, implicit user feedback, and generative AI to deliver highly personalized job matches and tailored resumes.

Generic job boards return too many broad keyword matches, ignoring strict user constraints like salary floors, location, visa sponsorship, seniority exclusions, and career pivots. JobPilot translates these constraints into a deterministic ranking pipeline, rigorously tested against four distinct edge-case personas.

2. System Architecture & Pipeline Design

JobPilot follows a modular architecture that combines offline batch data, on-demand live API ingestion, and generative AI tasks.

 Architecture Diagram

2.1 Hybrid Data Ingestion

- 1. Offline Baseline (Snapshot):** Seeded with 30,000 structured LinkedIn job postings (`jobs_snapshot.csv`). This guarantees a rich vector space for offline testing without relying on external APIs.
- 2. Live Adzuna Ingestion:** The sidebar **Fetch New Jobs** action pulls current postings from Adzuna on demand. A deduplication layer drops identical postings before writing to SQLite and dynamically updating the FAISS index. The codebase also includes a Kafka-style producer/consumer queue to demonstrate the streaming pattern, while the deployed UI uses the more reliable one-shot fetch for grading demos.

2.2 Generative AI Profile Parsing & Tailored Resumes

Raw PDF/DOCX resumes are parsed using Gemini Flash (`gemini-flash-latest`) with strict JSON schema enforcement to extract unstructured text into structured arrays (skills, target roles, dealbreakers). The **"Generate Resume"** engine utilizes RAG principles. By feeding persona constraints (e.g., "no defense," "new grad") alongside the retrieved job description, Gemini dynamically rewrites the candidate's resume in Markdown to perfectly align with the target role while adhering to pass criteria.

3. BAX-423 Technique Choices & Implementations

JobPilot integrates three core concepts from the BAX-423 curriculum to outperform standard baseline models.

3.1 Dense Embedding Retrieval vs. Sparse Keyword Search (Lecture 5)

Traditional ATS rely on BM25 sparse keyword retrieval, suffering from extreme vocabulary mismatch. A candidate highlighting "Data Visualization and Pandas" might fit an "Analytics Engineer" role perfectly, yet a BM25 system ranks them poorly without exact keyword overlap.

- **Technique:** We implemented Dense Embedding Retrieval using HuggingFace's `sentence-transformers/all-MiniLM-L6-v2`. This maps resumes and 30,000+ jobs into a shared 384-dimensional dense vector space, capturing deep semantic intent.
- **Engine:** We utilize **FAISS** with an `IndexFlatL2` index to perform highly efficient nearest-neighbor searches, retrieving candidate matches in milliseconds.

3.2 Multi-Stage Ranking Pipeline (Lecture 7)

While dense embeddings excel at semantic recall, they cannot enforce discrete business rules (e.g., a \$50k salary is a dealbreaker for a user demanding \$120k).

- **Technique:** We engineered a strict Multi-Stage Funnel:
 1. **L1 (Recall):** FAISS retrieves the top 1000 semantic matches.
 2. **L2 (Hard Filtering):** Deterministic boolean logic eradicates any jobs violating explicit parameters (Seniority markers, Visa Status, Salary thresholds).
 3. **L3 (Weighted Re-Ranking):** Surviving jobs are re-scored using a linear formula balancing semantics, exact skill overlap, and location: $\text{Final Score} = 0.25 * \text{Semantic} + 0.25 * \text{Skill} + 0.25 * \text{Role} + 0.10 * \text{Location} + 0.15 * \text{Feedback}$

3.3 Adaptive Semantic Feedback (Lecture 8)

A static ranking system cannot learn a user's unstated or evolving preferences.

- **Technique:** We built an Implicit Feedback Loop. When a user clicks  **Accept** or  **Pass** on a job card, the system logs the interaction into SQLite. The Re-Ranker dynamically recalculates, applying a Semantic Penalty Array to penalize related companies, job titles, and experience levels in real-time.

4. Test Personas & Rigorous Evaluation

We benchmarked the Multi-Stage pipeline (L7) against Embedding retrieval (L5) and a traditional BM25 keyword baseline, measuring **Precision@10** and **NDCG@10**. Relevance labels are persona-aware heuristics: a job is counted as relevant only when it matches the persona's target role pattern and does not violate that persona's hard constraints.

4.1 Persona Profiles & Pass/Fail Status

Persona	Key Pass Criteria	Edge Case & Implementation Notes	Status
Aisha (Pivoter)	Top-10 has zero Senior/Staff roles, zero defense companies. All ML-related.	L2 Hard Filter: Successfully parsed and blocked Senior/Staff and defense industry flags.	 PASS
Marcus (New Grad)	No 3+ yr requirements. Resume leads with education.	Adaptive Feedback: Learns preference, heavily penalizing mid-senior roles after rejection. Resume gen enforces education-first.	 PASS
Priya (Niche)	No Junior titles, Kafka/Spark framed as ML infrastructure.	L3 Re-ranker: The 25% Skill Overlap weight corrected embedding over-generalization. GenAI reframes backend skills as ML Ops.	 PASS
Kenji (Visa)	No contract roles, favors large companies or research labs.	Transparent Proxy System: The hard filter removes contract/temp/1099 roles and explicit no-sponsorship wording. L3 boosts known large employers, likely sponsors, universities, and research labs because the dataset lacks a guaranteed sponsorship field.	 PASS

4.2 Offline Benchmark Results Comparison

The benchmark shows two complementary outcomes: all four personas pass their required rule-based criteria, while P@10 and NDCG@10 remain realistic because they are computed from persona-aware heuristic relevance labels rather than manually inflated scores.

Persona	BM25 P@10	BM25 NDCG@10	Embed P@10	Embed NDCG@10	Multi- Stage P@10	Multi-Stage NDCG@10
Aisha	0.30	0.26	0.20	0.22	1.00	1.00
Marcus	0.30	0.43	0.20	0.17	1.00	1.00
Priya	0.20	0.15	0.50	0.42	0.10	0.22
Kenji	0.50	0.65	0.50	0.63	0.70	0.76

(P@10 = Precision of top 10 results. NDCG@10 = Normalized Discounted Cumulative Gain. Aisha and Marcus score highly because their constraints can be cleanly enforced with hard filters. Priya remains lower because MLOps/company-size relevance is sparse and partly proxy-based in the dataset. Kenji improves under the multi-stage pipeline because large-company/research-lab signals and contract/sponsorship filters help rank safer visa-compatible options.)

4.3 UI & "Explain" Feature Transparency

The Streamlit interface includes an **Explain feature** ("Why ranked here?") for every matched job, breaking down exactly how the 0-100% Match Score was calculated. It explicitly lists which Pass Criteria

passed or failed, which specific skills matched, and how recent Like/Pass feedback influenced the final mathematical score.

5. System Limitations & Future Enhancements

1. **Embedding Context Window Truncation:** The `all-MiniLM-L6-v2` transformer enforces a strict 256-token limit. Many enterprise job descriptions exceed 1,000 words. Consequently, specific technical requirements located at the bottom of a posting may be truncated.
 - *Solution:* Implement a sliding window document chunking strategy with Max-Pooling, or upgrade to a modern long-context embedding model such as `Nomic-Embed`.
2. **Company Size and Visa Sponsorship Proxies:** The underlying Kaggle dataset lacks native fields for exact company headcount and H1-B sponsorship. JobPilot implements transparent proxies (e.g., detecting "seed" or "stealth" for tiny startups, filtering explicit "no sponsorship" language, and boosting known large employers/research labs). These heuristics are intentionally shown as proxies rather than treated as guaranteed company-size or visa data.
 - *Solution:* Integrate third-party API enrichments (e.g., Clearbit or MyVisaJobs) during the ingestion pipeline to append hard data tags.
3. **Feedback Sparsity and the Cold Start Problem:** The adaptive semantic feedback engine performs exceptionally well after a few interactions, but brand-new users face a "cold start" relying solely on zero-shot embeddings.
 - *Solution:* Implement Cross-User Collaborative Filtering. By analyzing interaction histories of users with similar demographic profiles, the system can dynamically assign default interaction weights before the first click.
4. **FAISS Index Maintenance:** The deployed app loads a prebuilt FAISS index for the 30,000-row offline snapshot and can append newly fetched Adzuna jobs during the current session. As the SQLite database grows substantially, periodic offline rebuilds or a more advanced persistent incremental index would be better.
 - *Solution:* Transition to an incremental `IndexIVFFlat` FAISS architecture, allowing for persistent batch vector additions without full re-computation.